

计算机网络

一、网络基础

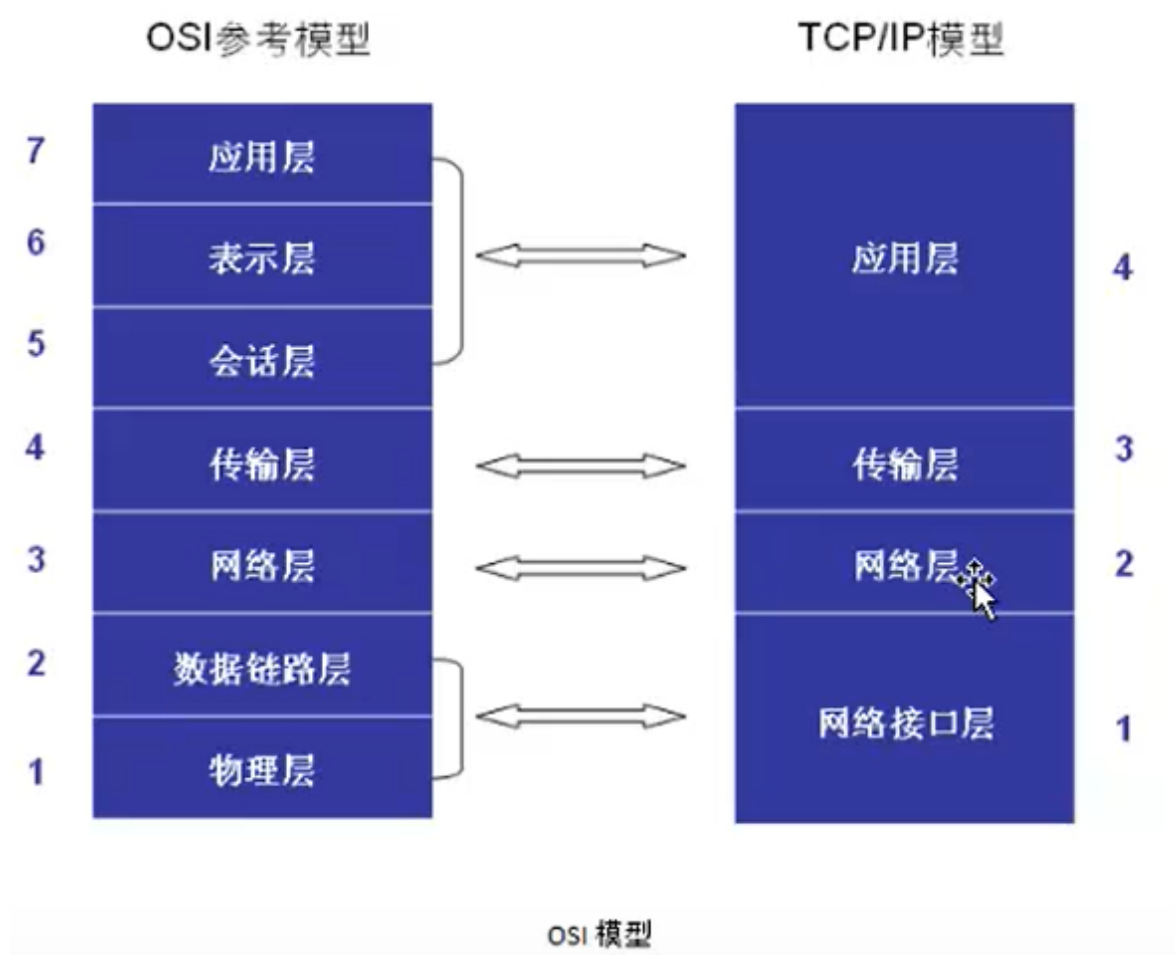
1、协议概念

协议，网络协议的简称，网络协议是通信计算机双方必须共同遵从的一组约定。协议可以理解为规则，有：原始协议、标准协议。

比如A、B之间传输文件，第一次传文件名、第二次传文件大小、第三次传文件内容，按照这个步骤进行传输。例如：TCP协议、UDP协议、HTTP协议、FTP协议等

2、分层模型

OSI的七层协议、TCP/IP四层协议



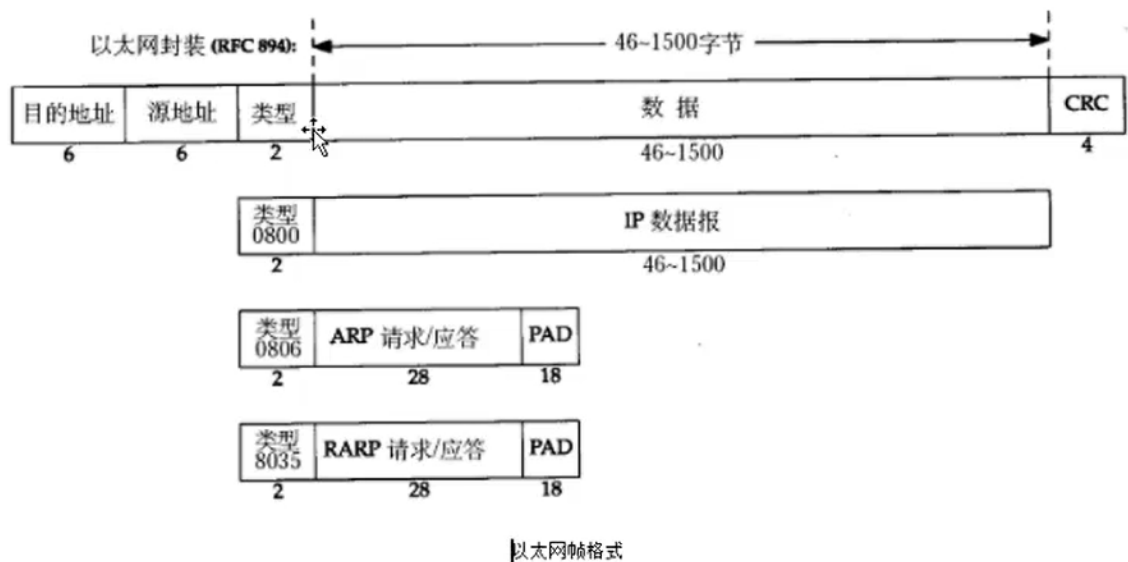
3、协议格式

链路层的以太网帧（帧头与帧尾）、网络层的IP数据报、传输层的数据段（这三个是由操作系统进行操作的）、应用层是由用户操作的。

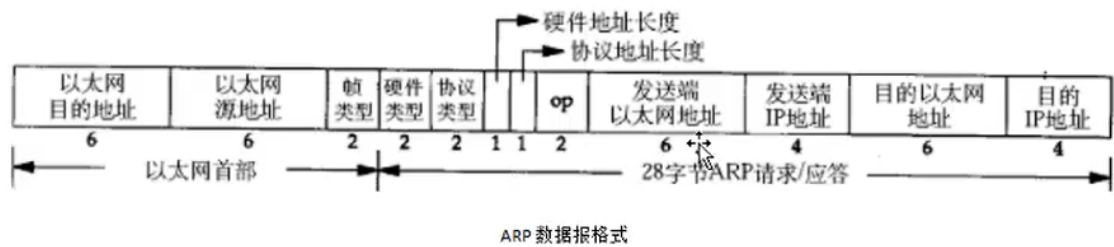
3.1、以太网帧格式

mac地址：也就是硬件地址，网卡的编号，表示网卡的唯一性。

以太网的帧格式如下所示：



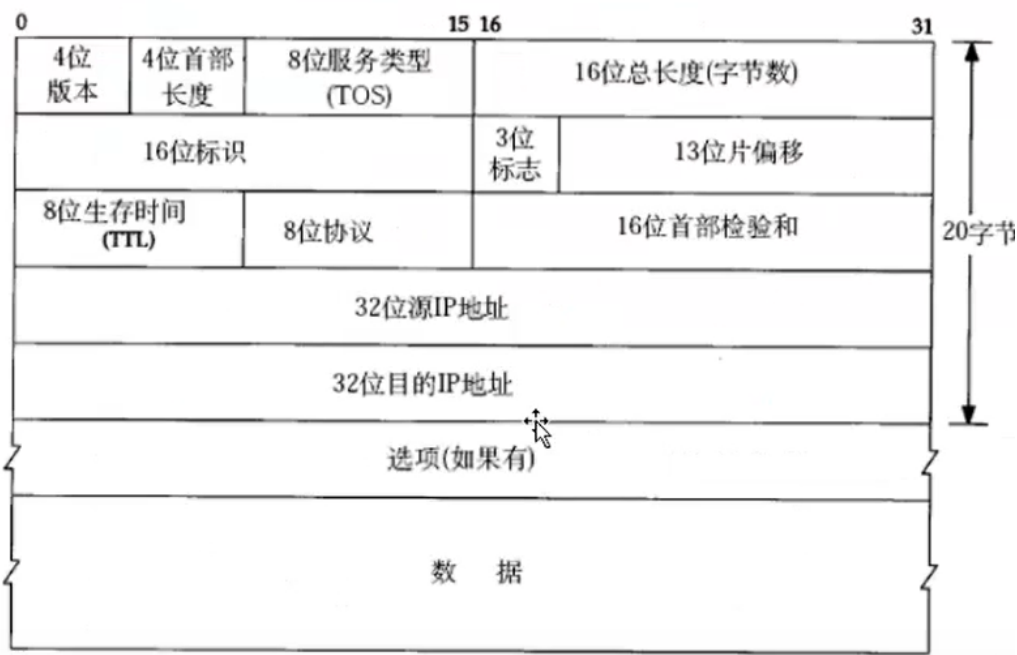
ARP 数据报的格式如下所示：



arp数据报就是为了获取下一跳的mac地址。

3.2、IP段格式

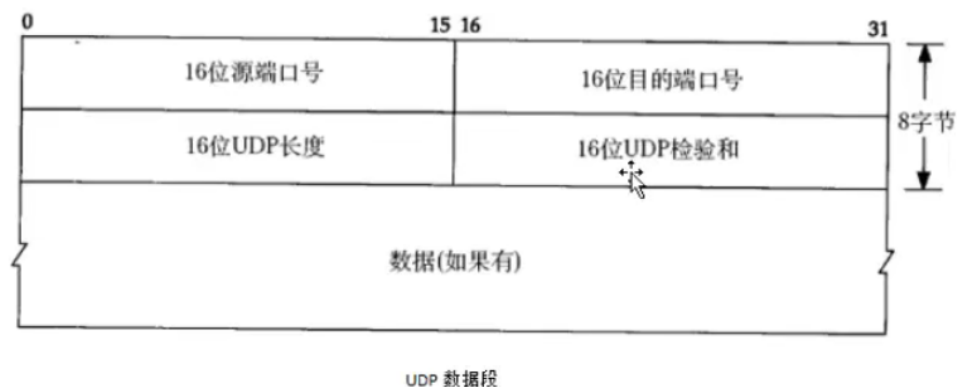
IP 段格式



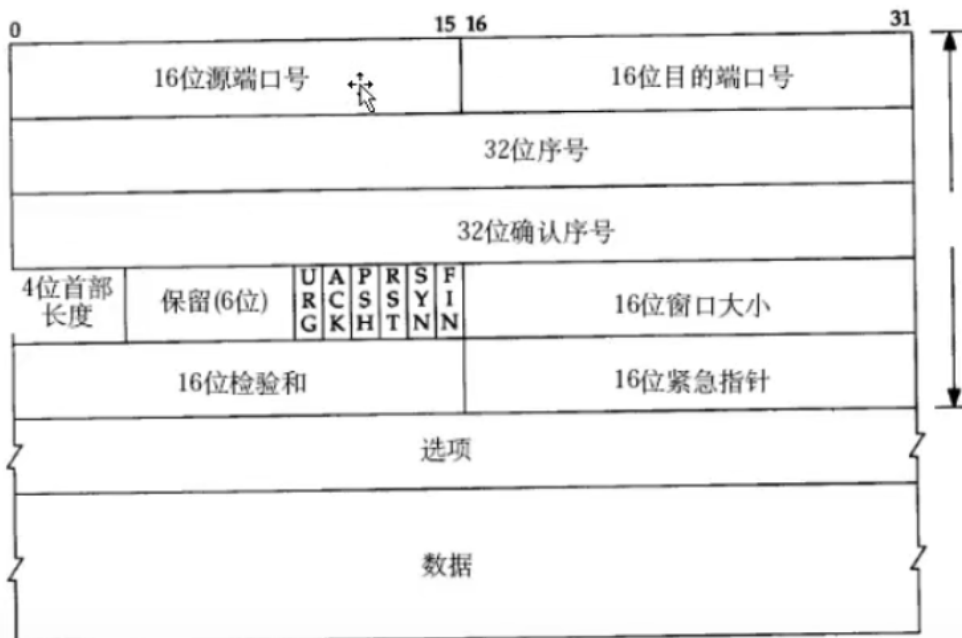
IP数据报的首部长度与数据长度都是可变的，但总是4的倍数。另外，4位版本号，也就是IPv4与IPv6，4位首部长度，就是IP数据报的首部长度，最小是20字节，最大是60字节，因为4位首部长度最大表示的是 $2^4-1=15$ ， $4 * 15$ 也就是60字节。8位的TTL，也就是下一跳的个数，最大也就是255，然后就是32位的源IP地址与32位的目的IP地址，也就是源IP与目的IP都占4字节。

3.3、TCP/UDP数据报格式

UDP 数据报格式



TCP 数据报格式



4、TCP协议

TCP协议是一个传输层的协议；TCP是面向连接的协议；TCP是一个可靠的协议；TCP是全双工的协议；TCP是一个字节流的协议；TCP是一个可以进行流量控制的协议

三次握手

TCP建立连接的过程叫做握手，握手需要在客户端和服务端之间交换三个TCP报文段。三次握手的目的是为了建立可靠连接。

第一次握手：客户端给服务器发送一个同步报文段SYN，并指定客户端的初始序列号ISN，此时客户端处于SYN_SENT状态。首部的同步位SYN = 1(SYN只是一个比特位，0表示不是SYN，1表示是SYN)，初始序列号seq = m。SYN = 1的报文段不能携带任何的数据，但要指定序号。

第二次握手：服务器接收到来自客户端的同步报文段SYN后，会以自己的SYN报文作为应答，并且也指定了自己的初始序列号ISN。同时会把客户端的 seq+1 确认序列号ack的值，表示自己已经收到了客户端的同步报文段SYN，此时服务器处于SYN_RCVD的状态。确认报文段中SYN = 1,ACK = 1(ACK也只是一个比特位，0表示不是ACK，1表示是ACK)，确认序列号ack = m+1，初始序列号seq = n。

第三次握手：客户端收到来自服务器的同步报文段SYN之后，会发送一个确认报文段ACK，以服务器的seq+1作为ack的值，表明自己已经收到来自服务器的同步报文段SYN。客户端进入ESTABLISHED状态，服务器确认报文段ACK之后，也会进入ESTABLISHED状态。确认报文段中，ACK = 1，确认序列号ack = n+1，序列号seq = m+1。

双方已经建立起连接，可以正常的发送数据。

四次挥手

TCP的连接释放。由客户端到服务器需要一个FIN和ACK，再由服务器到客户端需要一个FIN和ACK，因此通常被称为四次握手。在断开连接之前客户端和服务器都处于ESTABLISHED状态，双方都可以主动断开连接，以客户端主动断开连接为优。

第一次挥手：客户端打算断开连接，向服务器发送FIN报文(FIN标记位被设置为1，1表示为FIN，0表示不是)，FIN报文中会指定一个序列号，之后客户端进入FIN_WAIT_1状态。也就是客户端发出连接释放报文段(FIN报文)，指定序列号seq = u，主动关闭TCP连接，等待服务器的确认。

第二次挥手：服务器收到连接释放报文段(FIN报文)后，就向客户端发送ACK应答报文，以客户端的FIN报文的序列号 seq+1 作为ACK应答报文段的确认序列号ack = seq+1 = u + 1。接着服务器进入CLOSE_WAIT(等待关闭)状态，此时的TCP处于半关闭状态(下面会说什么是半关闭状态)，客户端到服务器的连接释放。客户端收到来自服务器的ACK应答报文段后，进入FIN_WAIT_2状态。

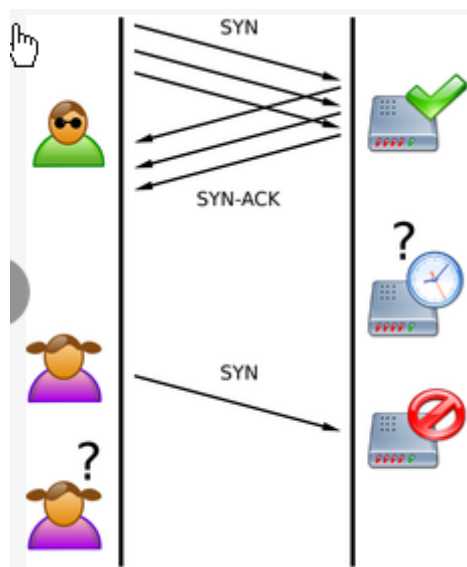
第三次挥手：服务器也打算断开连接，向客户端发送连接释放(FIN)报文段，之后服务器进入LASK_ACK(最后确认)状态，等待客户端的确认。服务器的连接释放(FIN)报文段的FIN=1，ACK=1，序列号seq=m，确认序列号ack=u+1。

第四次挥手：客户端收到来自服务器的连接释放(FIN)报文段后，会向服务器发送一个ACK应答报文段，以连接释放(FIN)报文段的确认序号 ack 作为ACK应答报文段的序列号 seq，以连接释放(FIN)报文段的序列号 seq+1作为确认序号ack。之后客户端进入TIME_WAIT(时间等待)状态，服务器收到ACK应答报文段后，服务器就进入CLOSE(关闭)状态，到此服务器的连接已经完成关闭。

SYN请求建立连接的标志，ACK应答的标志。

SYN攻击

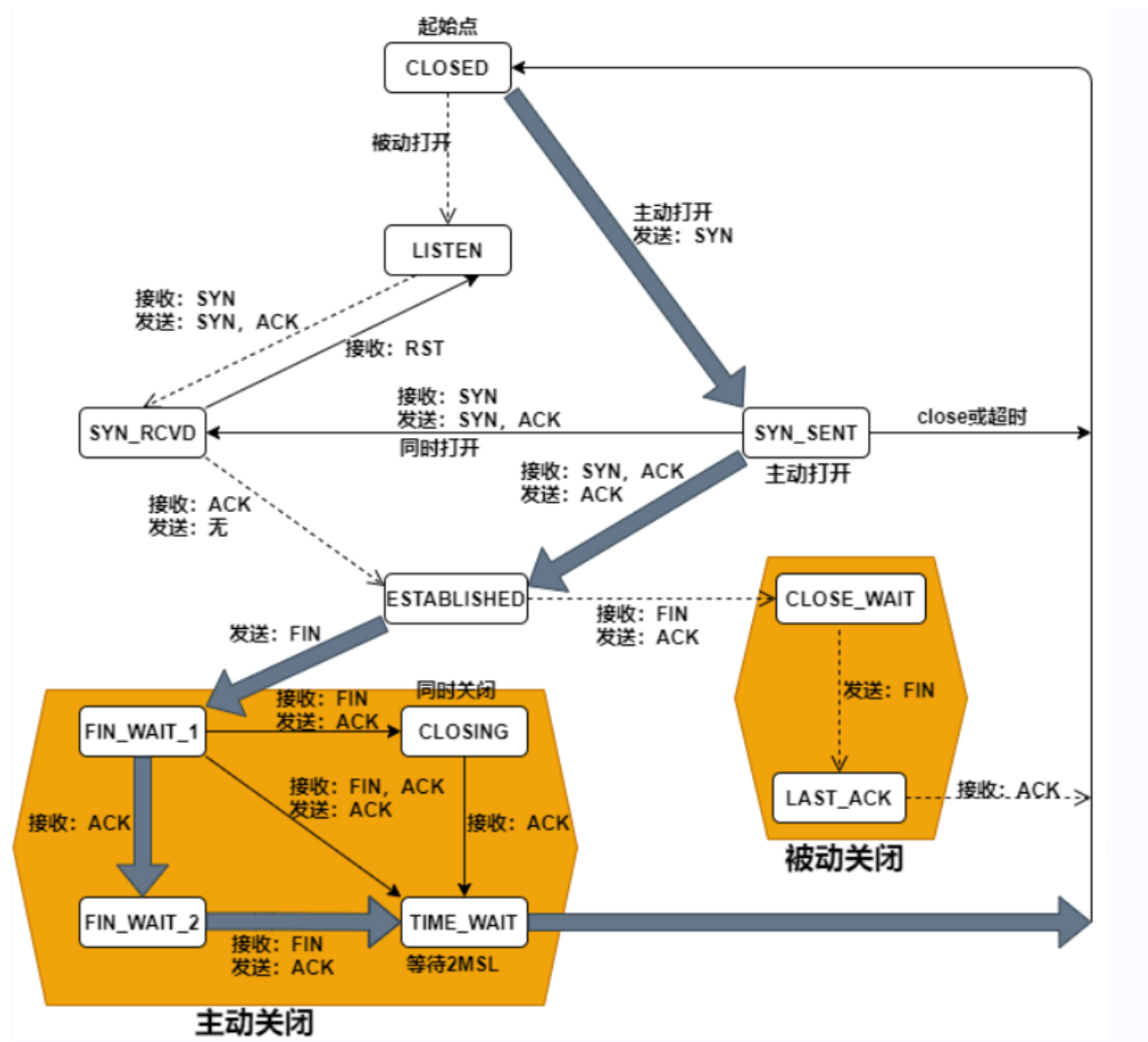
SYN攻击就是Client在短时间内伪造大量不存在的IP地址，并向Server不断地发送SYN包，Server回复确认包，并等待Client的确认，由于源地址是不存在的，因此，Server需要不断重发直至超时，这些伪造的SYN包将产时间占用未连接队列（内核会为每个这样的连接分配资源的），导致正常的SYN请求因为队列满而被丢弃，从而引起网络堵塞甚至系统瘫痪。SYN攻击是一种典型的DDOS攻击，检测SYN攻击的方式非常简单，即当Server上有大量半连接状态且源IP地址是随机的，则可以断定遭到SYN攻击了。



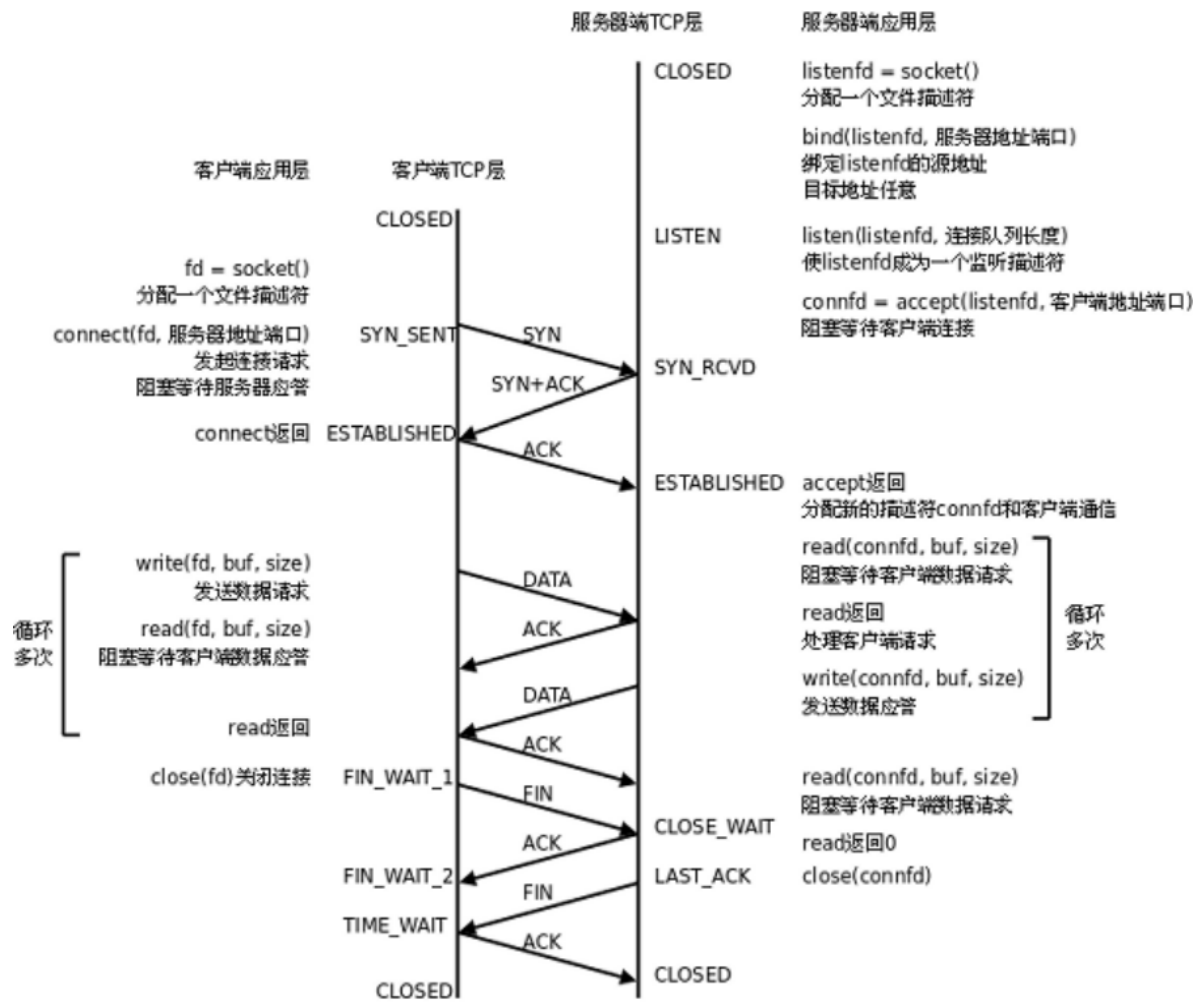
5、状态迁移图的解析

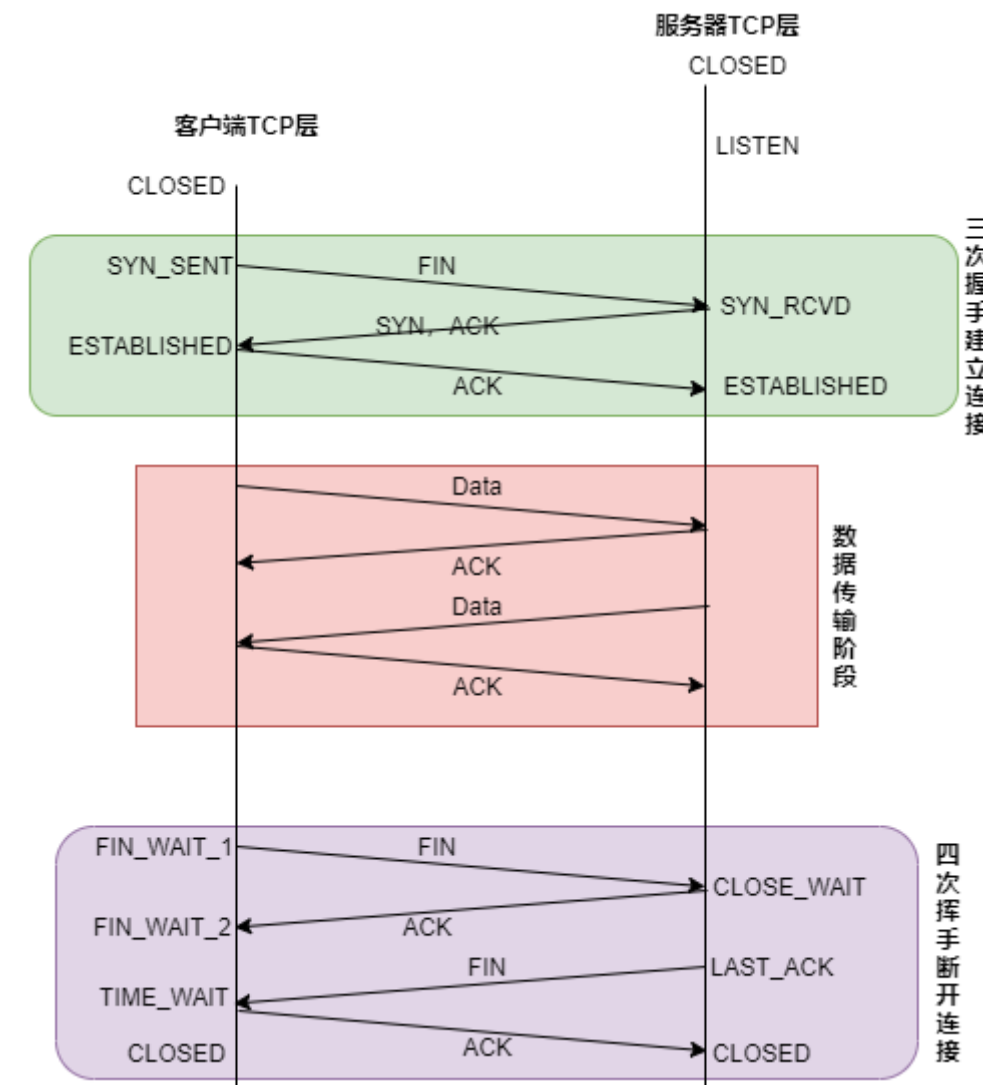
粗实线：主动发起连接与主动关闭连接；虚线：被动发起连接与被动关闭连接；细实线，两端同时操作的部分。

为何主动发起关闭的一端要设置TIME_WAIT的状态：确保最后一个ACK可以顺利到达对端。



6、TCP通信状态与程序结合分析





二、网络编程

1、基础

IP地址可以在网络环境中唯一标识一台主机。端口号可以在主机中唯一标识一个进程。所以在网络环境中唯一标识一个进程可以使用IP地址与端口号Port。

2、字节序

TCP/IP协议规定，**网络数据流应采用大端字节序。**

大端：低地址存高位，高地址存低位；小端：低地址存低位，高地址存高位（x86采用小端存储）

网络字节序，就是在网络中进行传输的字节序列，采用的是大端法。主机字节序，就是本地计算机中存储数据采用的字节序列，采用的是小端法。

对应的函数

```
1 #include <arpa/inet.h>
2 uint32_t htonl(uint32_t hostlong); //h = host n = network l = long s = short
3 uint16_t htons(uint16_t hostshort);
4 uint32_t ntohl(uint32_t netlong);
5 uint16_t ntohs(uint16_t netshort);
```

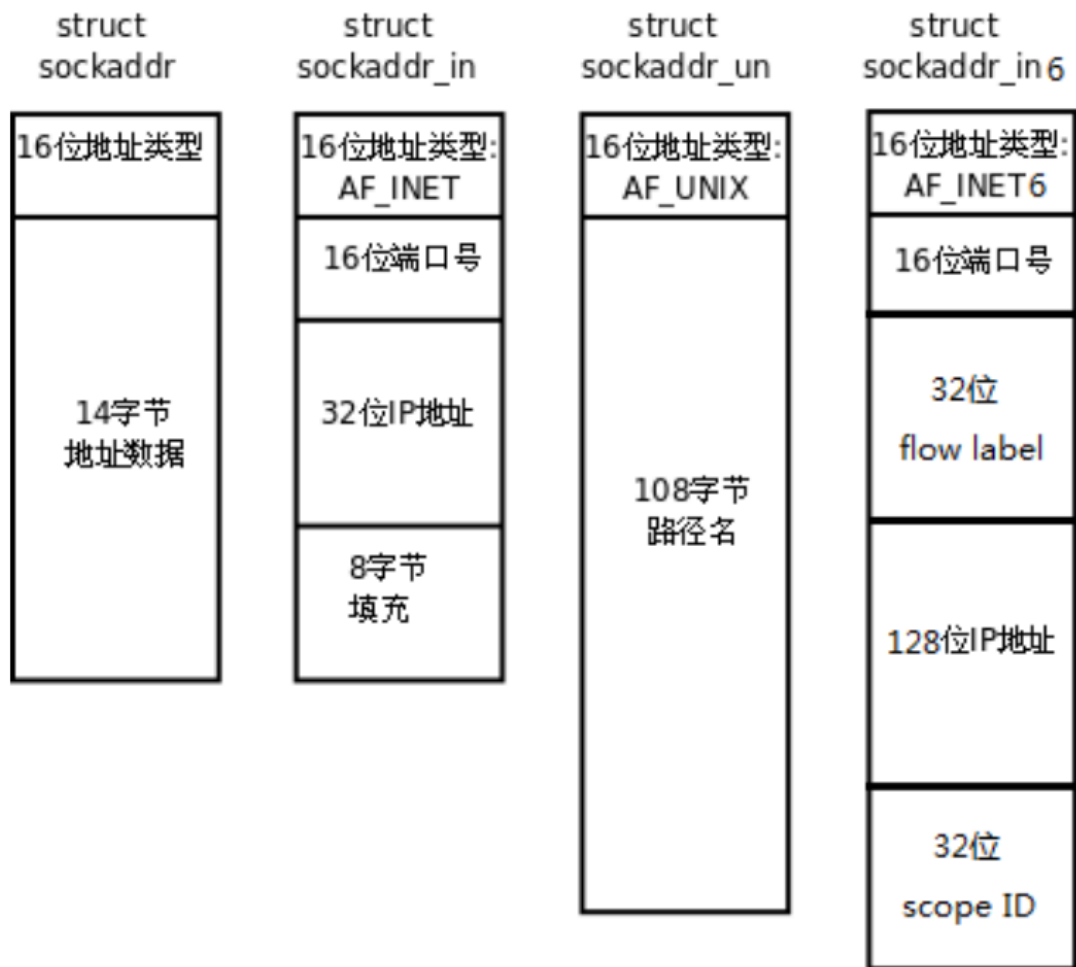
```

1  #include <arpa/inet.h>
2  //点分十进制字符串转换为网络字节序
3  int inet_pton(int af, const char *src, void *dst);
4
5  //网络字节序转换为点分十进制字符串
6  const char *inet_ntop(int af, const void *src, char *dst, socklen_t size);
7
8  #include <sys/socket.h>
9  #include <netinet/in.h>
10 #include <arpa/inet.h> //包含了<netinet/in.h>, 后者包含了<sys/socket.h>
11
12 typedef uint32_t in_addr_t;
13 struct in_addr
14 {
15     in_addr_t s_addr;
16 };
17
18 //将cp所指C字符串转换成一个32位的网络字节序二进制值, 并通过inp指针来存储, 成功返回1, 失败
19 //返回0
20 int inet_aton(const char *cp, struct in_addr *inp);
21
22 //将一个32位的网络字节序二进制IPv4地址转换成相应的点分十进制数串, 由该函数的返回值所指向的
23 //字符串驻留在静态内存中, 这意味着该函数是不可重入的
24 char *inet_ntoa(struct in_addr in);
25
26 //inet_addr函数转换网络主机地址 (如192.168.1.10) 为网络字节序二进制值, 如果参数char
27 // *cp无效, 函数返回-1(INADDR_NONE), 这个函数在处理地址为255.255.255.255时也返回-
28 // 1, 255.255.255.255是一个有效的地址, 不过inet_addr无法处理;
29 //返回值为32位的网络字节序二进制
30 in_addr_t inet_addr(const char *cp); //ok
31
32 in_addr_t inet_network(const char *cp);
33
34 struct in_addr inet_makeaddr(in_addr_t net, in_addr_t host);
35
36 in_addr_t inet_lnaof(struct in_addr in);
37
38 in_addr_t inet_netof(struct in_addr in);
39
40 #include <sys/socket.h>
41 int getsockname(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
42 int getpeername(int sockfd, struct sockaddr *addr, socklen_t *addrlen);

```

3、常用结构体

man 7 ip可以查看相应的结构体, 也可以使用 命令 `sudo grep -rn "struct sockaddr_in {" /usr` 进行搜索。



```

1 struct sockaddr
2 {
3     sa_family_t sa_family;    /* address family, AF_xxx */
4     char sa_data[14];        /* 14 bytes of protocol address */
5 };
6
7 struct sockaddr_in
8 {
9     sa_family_t sin_family; /* address family: AF_INET */
10    in_port_t sin_port;      /* port in network byte order */
11    struct in_addr sin_addr; /* internet address */
12 };
13
14 /* Internet address. */
15 struct in_addr
16 {
17     uint32_t s_addr;        /* address in network byte order */
18 };

```

IPv4和IPv6的地址格式定义在netinet/in.h中，IPv4地址用sockaddr_in结构体表示，IPv6地址使用sockaddr_in6结构体表示。UNIX Domain Socket的地址格式定义在sys/un.h中，使用sockaddr_un结构体表示。所有的地址类型分别定义为常数AF_INET、AF_INET6、AF_UNIX。

```

1 struct sockaddr_in addr;
2 addr.sin_family = AF_INET/AF_INET6/AF_UNIX;
3 addr.sin_port = htons/ntohs;
4 addr.sin_addr.s_addr = htonl/ntohl/inet_pton/inet_ntop

```

4、网络编程相关函数

4.1、socket函数

创建套接字

```
1  #include <sys/types.h>           /* See NOTES */
2  #include <sys/socket.h>
3  //创建套接字函数
4  int socket(int domain, int type, int protocol);
5  domain:AF_INET/AF_INET6/AF_UNIX
6  type:SOCK_STREAM/SOCK_DGRAM 前者默认是TCP, 后者默认是UDP
7  protocol:传0表示使用默认协议
8
9  //函数返回值
10 成功, 返回指向新创建的socket的文件描述符, 失败返回-1, 设置errno
```

4.2、bind函数

因为服务器程序所监听的网络地址与端口号是固定不变的, 所以需要使用bind函数进行绑定。bind函数将sockfd与addr绑定在一起, 使sockfd这个用于网络通讯的文件描述符监听addr所描述的地址和端口号。**绑定IP与端口号**

```
1  #include <sys/types.h>           /* See NOTES */
2  #include <sys/socket.h>
3  int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
4  //绑定服务的端口号与IP地址
5  sockfd:上面socket创建的套接字
6  addr: 所要绑定的ip地址与端口号
7  addrlen:前面addr结构体的长度
8
9  //函数返回值
10 成功, 返回指向新创建的socket的文件描述符, 失败返回-1, 设置errno
```

4.3、listen函数

用来指定监听上限数值(允许同时多少个客户端与服务器建立连接), **指定最大同时发起连接数。**

```
1  #include <sys/types.h>           /* See NOTES */
2  #include <sys/socket.h>
3
4  int listen(int sockfd, int backlog);
5  sockfd:socket创建的文件描述符
6  backlog:排队建立3次握手队列和刚刚建立3次握手队列的连接数和。
```

可以使用命令进行查看:

```
1  cat /proc/sys/net/ipv4/tcp_max_syn_backlog
```

4.4、accept函数

接收连接请求的函数, **阻塞等待客户端发起连接**

如果客户端还没有来得及连接, 此时accept函数会处于阻塞状态。

```

1  #include <sys/types.h>           /* See NOTES */
2  #include <sys/socket.h>
3
4  int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
5  sockfd: socket创建的文件描述符
6  addr: 传出参数, 返回连接客户端地址信息, 包含IP地址与端口号
7  addrlen: 传入传出参数 (值-结果), 传入sizeof(addr)大小, 函数返回时返回真正接收到地址结构体的大小。
8
9  //函数返回值
10 成功返回一个新的socket文件描述符, 用于和客户端通信, 失败返回-1, 设置errno

```

4.5、connect函数

客户端调用该函数, 连接到服务器上。**发起连接**

```

1  #include <sys/types.h>           /* See NOTES */
2  #include <sys/socket.h>
3
4  int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
5  sockfd: 是客户端自己使用socket得到的文件描述符。
6  addr: 传入参数, 指定服务器端地址信息, 包含IP地址与端口号
7  addrlen: 传入参数, 传入sizeof(addr)大小
8
9  返回值: 成功返回0, 失败返回-1, 设置errno

```

客户端需要调用connect连接服务器, connect和bind的采纳数形式一致, 区别在于bind的参数是自己的地址, 而connect的参数是对方的地址。

4.6、close函数

关闭套接字创建的文件描述符。

```

1  #include <unistd.h>
2  int close(int fd);

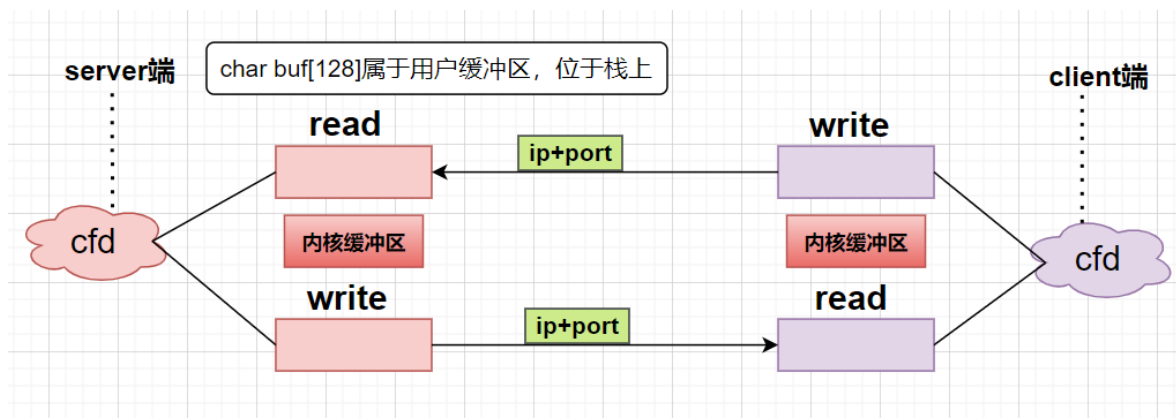
```

客户端其实也是需要bind端口号与IP地址, 如果没有显示绑定的话, 操作系统会自动分配一个IP地址与端口号。但是服务器是不能不使用bind函数, 让操作系统随机分配IP地址与端口号, 这样的话客户端就不知道服务器的IP地址与端口号, 就不知道怎么连接到服务器上了, 也不知道连接到那个服务器上。

本地随机的有效数字类型的IP, INADDR_ANY。

INADDR_ANY解析: 转换过来就是0.0.0.0, 泛指本机的意思, 表示本机的所有IP, 因为有些电脑不止一块网卡, 如果某个应用程序只监听某个端口, 那么其他端口过来的数据就接收不了。

5、网络编程代码



5.1、端口复用

让同一个端口可以进行重复使用, 不至于等待2MSL的时间

```

1 #include <sys/types.h>
2 #include <sys/socket.h>
3 int getsockopt(int sockfd, int level, int optname, void *optval, socklen_t
  *optlen);
4
5 int setsockopt(int sockfd, int level, int optname,
6               const void *optval, socklen_t optlen);

```

```

1 int opt = 1;
2 setsockopt(listenfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
3
4 int opt = 1;
5 setsockopt(listenfd, SOL_SOCKET, SO_REUSEPORT, &opt, sizeof(opt));

```

5.2、服务器端源码

```

1 #include <stdio.h>
2 #include <sys/types.h>
3 #include <sys/socket.h>
4 #include <arpa/inet.h>
5 #include <unistd.h>
6 #include <ctype.h>
7
8 #define SERV_IP "127.0.0.1"
9 #define SERV_PORT 6666
10
11 int main()
12 {
13     int sfd, cfd;
14     struct sockaddr_in serv_addr, clie_addr;
15     socklen_t clie_addr_len;
16     char buf[BUFSIZ], clie_IP[BUFSIZ];
17     int nByte, idx;
18
19     sfd = socket(AF_INET, SOCK_STREAM, 0);
20
21     int opt = 1;

```

```

22     setsockopt(sfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)); //允许端口复
    用
23
24     memset(&serv_addr, 0, sizeof(serv_addr));
25     serv_addr.sin_family = AF_INET;
26     serv_addr.sin_port = htons(SERV_PORT);
27     serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
28
29     bind(sfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr));
30
31     listen(sfd, 128);
32
33     clie_addr_len = sizeof(clie_addr);
34     cfd = accept(sfd, (struct sockaddr *)&clie_addr, &clie_addr_len);
35     printf("client IP: %s, port: %d\n",
36           inet_ntop(AF_INET, &clie_addr.sin_addr.s_addr, clie_IP,
    sizeof(clie_IP)),
37           ntohs(clie_addr.sin_port));
38
39     while(1)
40     {
41         nByte = read(cfd, buf, sizeof(buf));
42
43         for(idx = 0; idx < nByte; ++idx)
44         {
45             buf[idx] = toupper(buf[idx]);
46         }
47
48         write(cfd, buf, nByte);
49     }
50
51     close(sfd);
52     close(cfd);
53     return 0;
54 }

```

5.3、客户端源码

```

1  #include <stdio.h>
2  #include <sys/types.h>
3  #include <sys/socket.h>
4  #include <arpa/inet.h>
5  #include <unistd.h>
6  #include <ctype.h>
7  #include <string.h>
8
9  #define SERV_IP "127.0.0.1"
10 #define SERV_PORT 6666
11
12 int main()
13 {
14     int cfd;
15     struct sockaddr_in serv_addr;
16     char buf[BUFSIZ];
17     int nByte;
18
19     cfd = socket(AF_INET, SOCK_STREAM, 0);

```

```

20
21     memset(&serv_addr, 0, sizeof(serv_addr));
22     serv_addr.sin_family = AF_INET;
23     serv_addr.sin_port = htons(SERV_PORT);
24     serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
25     /* inet_pton(cfd, SERV_IP, &serv_addr.sin_addr.s_addr); */
26
27     connect(cfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr));
28
29     while(1)
30     {
31         fgets(buf, sizeof(buf), stdin); //hello world ----> hello world\n\n0
32         write(cfd, buf, strlen(buf));
33
34         nByte = read(cfd, buf, sizeof(buf));
35         write(STDOUT_FILENO, buf, nByte);
36     }
37
38     close(cfd);
39     return 0;
40 }

```

5.4、read返回值

1、大于0，实际读到的字节数，并且buf=1024

- 如果read读到的数据的长度等于buf，返回的就是1024
- 如果read读到的数据长度小于buf，那就是小于1024的数值。

2、返回值为0，数据读完（读到文件、管道、socket末尾 ---对端关闭）

2、返回值为0，数据读完（读到文件、管道、socket末尾 ---对端关闭）

3、返回值为0，数据读完（读到文件、管道、socket末尾 ---对端关闭） 3、返回值为-1，表明出现异常

3、返回值为-1，表明出现异常

- errno == EINTR 说明被信号中断 所以需要重启或者退出
- errno == EAGAIN (EWOULDBLOCK) 非阻塞方式读，并且没有数据
- 其他值 真的出现错误 perror打印 exit

5.5、readn/writen函数的封装

因为以太网帧一次只能传送1500字节的数据，所以使用read函数一次最多只能读到1500字节，就返回退出。

```

1  ssize_t readn(int fd, void *vptr, size_t n)
2  {
3      size_t nleft; //unsigned int 剩余未读取的字节数
4      size_t nread; //int 实际读到的字节数
5      char *ptr;
6
7      nleft = n; //n未读取字节数
8      ptr = vptr;
9
10     while(nleft > 0)
11     {
12         if((nread = read(fd, ptr, nleft)) < 0)
13         {

```

```

14         if(errno == EINTR)
15         {
16             nread = 0;
17         }
18         else
19         {
20             return -1;
21         }
22     }
23     else if(0 == nread)
24     {
25         break;
26     }
27
28     nleft -= nread;
29     ptr += nread;
30 }
31
32 return (n - nleft);
33 }

```

```

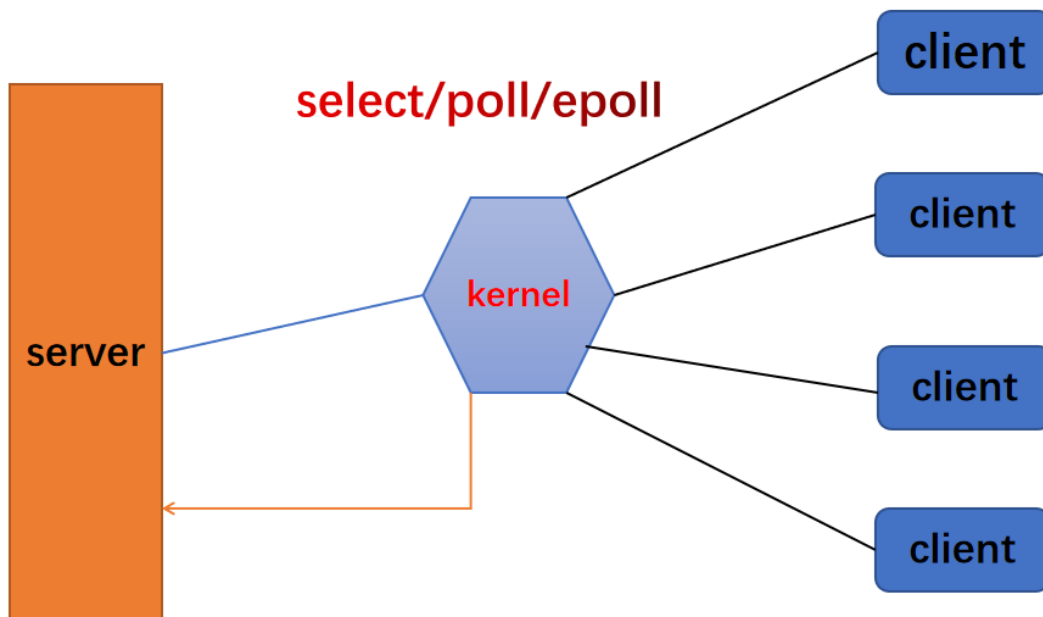
1  ssize_t writen(int fd, const void *vptr, size_t n)
2  {
3      size_t nleft;
4      size_t nwritten;
5      const char *ptr;
6
7      nleft = n;
8      ptr = vptr;
9
10     while(nleft > 0)
11     {
12         if((nwritten = write(fd, ptr, nleft)) <= 0)
13         {
14             if(nwritten < 0 && errno == EINTR)
15             {
16                 nwritten = 0;
17             }
18             else
19             {
20                 return -1;
21             }
22         }
23
24         nleft -= nwritten;
25         ptr += nwritten;
26     }
27
28     return n;
29 }

```

三、IO多路复用

1、概念与原理图

多进程与多线程并发服务器，不经常使用这种作为大型服务器开发的原因是，所有的监听与访问请求都由服务器操作。可以使用多路IO转接服务器（也叫多任务IO服务器），思想：不再由应用程序自己监视客户端连接，取而代之由内核替应用程序监视文件。



2、select

2.1、接口解析

```
1 #include <sys/select.h>
2 #include <sys/time.h>
3 #include <sys/types.h>
4 #include <unistd.h>
5
6 int select(int nfds, fd_set *readfds, fd_set *writefds, fd_set *exceptfds,
7           struct timeval *timeout);
8
9 nfds: 监控的文件描述符集里最大文件描述符+1, 因为此参数会告诉内核检测前多少个文件描述符的状态。
10 readfds/writes/exceptfds: 监控有读数据/写数据/异常发生到达文件描述符集合, 三个都是传入传出参数。
11 timeout: 定时阻塞监控时间, 3中情况:
12     1、NULL, 永远等下去
13     2、设置timeval, 等待固定时间
14     3、设置timeval里时间均为0, 检查描述字后立即返回, 轮询。
15
16 fd_set: 本质是个位图。
17
18 struct timeval
19 {
20     long    tv_sec;           /* seconds */
21     long    tv_usec;         /* microseconds */
22 };
23 返回值:
24     成功: 所监听的所有的监听集合中, 满足条件的总数。
25     失败: 返回-1.
```



```

1 void FD_ZERO(fd_set *set); //将set清空为0
2 void FD_SET(int fd, fd_set *set); //将fd设置到set集合中
3 void FD_CLR(int fd, fd_set *set); //将fd从set中清除出去
4 int FD_ISSET(int fd, fd_set *set); //判断fd是否在集合中

```

2.2、优缺点

- 1、文件描述符上限（1024），同时监听的文件描述符1024个，历史原因，不好修改，除非重新编译Linux内核。
- 2、当监听的文件描述符个数比较稀疏的时候（比如3，600，1023），循环判断比较麻烦，所以需要自定义数据结构：数组
- 3、监听集合与满足监听条件的集合是同一个，需要将原有集合保存。

2.3、代码实现（C语言）

server端

```

1  #include <stdio.h>
2  #include <sys/types.h>
3  #include <sys/socket.h>
4  #include <arpa/inet.h>
5  #include <sys/select.h>
6  #include <sys/time.h>
7  #include <stdlib.h>
8  #include <strings.h>
9  #include <string.h>
10 #include <unistd.h>
11 #include <ctype.h>
12
13 #define SERV_PORT 8888
14
15 int main()
16 {
17     int listenfd, connfd, sockfd;
18     struct sockaddr_in serv_addr, clie_addr;
19     socklen_t clie_addr_len;
20     int ret, maxfd, maxi, i, j, nready, nByte;
21     fd_set rset, allset;
22     int client[FD_SETSIZE];
23     char buf[BUFSIZ], str[BUFSIZ];
24
25     listenfd = socket(AF_INET, SOCK_STREAM, 0);
26     if(-1 == listenfd)
27     {
28         perror("socket error");
29         exit(-1);
30     }
31
32     int opt = 1;
33     setsockopt(listenfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
34
35     bzero(&serv_addr, sizeof(serv_addr));
36     serv_addr.sin_family = AF_INET;
37     serv_addr.sin_port = htons(SERV_PORT);
38     serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
39
40     ret = bind(listenfd, (struct sockaddr *)&serv_addr,
41               sizeof(serv_addr));

```

```

41     if(-1 == ret)
42     {
43         perror("bind error");
44         exit(-1);
45     }
46
47     ret = listen(listenfd, 128);
48     if(-1 == ret)
49     {
50         perror("listen error");
51         exit(-1);
52     }
53
54     maxfd = listenfd;
55
56     maxi = -1;
57     for(i = 0; i < FD_SETSIZE; ++i)
58     {
59         client[i] = -1;
60     }
61
62     FD_ZERO(&allset);
63     FD_SET(listenfd, &allset);
64
65     while(1)
66     {
67         rset = allset;
68         nready = select(maxfd + 1, &rset, NULL, NULL, NULL);
69         if(nready < 0)
70         {
71             perror("select error");
72             exit(-1);
73         }
74
75         if(FD_ISSET(listenfd, &rset))
76         {
77             clie_addr_len = sizeof(clie_addr);
78             connfd = accept(listenfd, (struct sockaddr *)&clie_addr,
&clie_addr_len);
79             if(-1 == connfd)
80             {
81                 perror("accept error");
82                 exit(-1);
83             }
84             printf("receive from %s from port %d\n",
85                 inet_ntop(AF_INET, &clie_addr.sin_addr, str,
sizeof(str)),
86                 ntohs(clie_addr.sin_port));
87
88             for(i = 0; i < FD_SETSIZE; ++i)
89             {
90                 if(client[i] < 0)
91                 {
92                     client[i] = connfd;
93                     break;
94                 }
95             }
96

```

```

97         if(i == FD_SETSIZE)
98         {
99             fputs("too many clients\n", stderr);
100             exit(1);
101         }
102
103         FD_SET(connfd, &allset);
104
105         if(connfd > maxfd)
106         {
107             maxfd = connfd;
108         }
109
110         if(i > maxi)
111         {
112             maxi = i;
113         }
114
115         if(--nready == 0)
116         {
117             continue;
118         }
119     }
120
121     for(i = 0; i <= maxi; ++i)
122     {
123         if((sockfd = client[i]) < 0)
124         {
125             continue;
126         }
127
128         if(FD_ISSET(sockfd, &rset))
129         {
130             if((nByte = read(sockfd, buf, sizeof(buf))) == 0)
131             {
132                 close(sockfd);
133                 FD_CLR(sockfd, &allset);
134                 client[i] = -1;
135             }
136             else if(nByte > 0)
137             {
138                 for(j = 0; j < nByte; ++j)
139                 {
140                     buf[j] = toupper(buf[j]);
141                 }
142                 write(sockfd, buf, nByte);
143                 write(STDOUT_FILENO, buf, nByte);
144             }
145
146             if(--nready == 0)
147             {
148                 break;
149             }
150         }
151     }
152 }
153
154 close(listenfd);

```

```

155     close(connfd);
156     return 0;
157 }

```

客户端

```

1  #include <stdio.h>
2  #include <sys/types.h>
3  #include <sys/socket.h>
4  #include <arpa/inet.h>
5  #include <unistd.h>
6  #include <ctype.h>
7  #include <string.h>
8  #include <stdlib.h>
9
10 #define SERV_IP "127.0.0.1"
11 #define SERV_PORT 8888
12
13 int main()
14 {
15     int cfd;
16     struct sockaddr_in serv_addr;
17     char buf[BUFSIZ];
18     int nByte;
19
20     cfd = socket(AF_INET, SOCK_STREAM, 0);
21     if(-1 == cfd)
22     {
23         perror("socket error");
24         exit(-1);
25     }
26
27     memset(&serv_addr, 0, sizeof(serv_addr));
28     serv_addr.sin_family = AF_INET;
29     serv_addr.sin_port = htons(SERV_PORT);
30     serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
31     /* inet_pton(cfd, SERV_IP, &serv_addr.sin_addr.s_addr); */
32
33     connect(cfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr));
34
35     while(1)
36     {
37         fgets(buf, sizeof(buf), stdin); //hello world ----> hello world\n\n0
38         write(cfd, buf, strlen(buf));
39
40         nByte = read(cfd, buf, sizeof(buf));
41
42         write(STDOUT_FILENO, buf, nByte);
43     }
44
45     close(cfd);
46     return 0;
47 }

```

3、poll

3.1、接口解析

```
1 #include <poll.h>
2 int poll(struct pollfd *fds, nfds_t nfds, int timeout);
3
4 struct pollfd
5 {
6     int fd;           /* file descriptor */
7     short events;      /* requested events */
8     short revents;     /* returned events */
9 };
10
11 fds:文件描述符数组。
12     events: POLLIN/POLLOUT/POLLERR
13 nfds: 监控数组中有多少文件描述符需要被监控。
14 timeout 毫秒级等待
15     -1:阻塞等, #define INFTIM -1 Linux中没有定义此宏
16     0:立即返回, 不阻塞进程
17     >0:等待指定毫秒数, 如当前系统时间精度不够毫秒, 向上取值。
18 函数返回值: 满足监听条件的文件描述符的数目。
```

3.2、优缺点

优点:

- 1、突破文件描述符1024的上限
- 2、监听与返回的集合分离
- 3、搜索范围变小 (已经知道是那几个数组)

缺点:

监听1000个文件描述符, 但是只有3个满足条件, 这样也需要全部遍历, 效率依旧低。

cat /proc/sys/fs/file-max 查看一个进程可以打开的文件描述符的上限数。

sudo vi /etc/security/limits.conf。在文件尾部写入以下配置,soft软限制, hard硬限制。

```
1 soft nfile 65536
2 hard nfile 100000
```

3.3、代码实现 (C语言)

server端

```
1 #include <stdio.h>
2 #include <sys/types.h>
3 #include <sys/socket.h>
4 #include <arpa/inet.h>
5 #include <unistd.h>
6 #include <ctype.h>
7 #include <string.h>
8 #include <stdlib.h>
9 #include <poll.h>
10 #include <errno.h>
11
12 #define SERV_PORT 8888
13 #define OPEN_MAX 1024
```

```

14
15 int main()
16 {
17     int i, j, n, maxi;
18     int nready, ret;
19     int listenfd, connfd, sockfd;
20     char buf[BUFSIZ], str[INET_ADDRSTRLEN];
21     struct sockaddr_in serv_addr, clie_addr;
22     socklen_t clie_addr_len;
23     struct pollfd client[OPEN_MAX];
24
25     listenfd = socket(AF_INET, SOCK_STREAM, 0);
26     if(-1 == listenfd)
27     {
28         perror("socket error");
29         exit(-1);
30     }
31
32     int opt = 1;
33     setsockopt(listenfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
34
35     memset(&serv_addr, 0, sizeof(serv_addr));
36     serv_addr.sin_family = AF_INET;
37     serv_addr.sin_port = htons(SERV_PORT); //本地字节序port与ip都要转换为网络字
节序
38     serv_addr.sin_addr.s_addr = htonl(INADDR_ANY); //因为要在网络上传输
39
40     ret = bind(listenfd, (struct sockaddr *)&serv_addr,
sizeof(serv_addr));
41     if(-1 == ret)
42     {
43         perror("bind error");
44         exit(-1);
45     }
46
47     ret = listen(listenfd, 128);
48     if(-1 == ret)
49     {
50         perror("listen error");
51         exit(-1);
52     }
53
54     client[0].fd = listenfd;
55     client[0].events = POLLIN;
56
57     for(i = 1; i < OPEN_MAX; ++i)
58     {
59         client[i].fd = -1; //将数组初始化为-1
60     }
61
62     maxi = 0;
63
64     while(1)
65     {
66         nready = poll(client, maxi + 1, -1);
67         if(nready < 0)
68         {
69             perror("poll error");

```

```

70         exit(-1);
71     }
72
73     if(client[0].revents & POLLIN)
74     {
75         clie_addr_len = sizeof(clie_addr);
76         connfd = accept(listenfd, (struct sockaddr *)&clie_addr,
193 &clie_addr_len); //立即连接, 此时不会阻塞等
77         if(-1 == connfd)
78         {
79             perror("accept error");
80             exit(-1);
81         }
82
83         printf("received from %s at port %d\n",
84             inet_ntop(AF_INET, &clie_addr.sin_addr.s_addr, str,
194 sizeof(str)),
85             ntohs(clie_addr.sin_port));
86
87         for(i = 1; i < OPEN_MAX; ++i)
88         {
89             if(client[i].fd < 0) //因为初始化为-1, 所以在此作为判断条件
90             {
91                 client[i].fd = connfd;
92                 break; //直接跳出, 免得继续判断, 浪费时间
93             }
94         }
95
96         if(i == OPEN_MAX) //select监听的文件描述符有上限, 最大只能监听1024个
97         {
98             fputs("too many clients\n", stderr);
99             exit(1);
100         }
101
102         client[i].events = POLLIN;
103
104         if(i > maxi)
105         {
106             maxi = i; //因为文件描述符有新增, 导致自定义数组有变化, 所以需要重新
195 修改maxi的值
107         }
108
109         if(--nready == 0) //意思不明确
110         {
111             continue;
112         }
113     }
114
115     for(i = 1; i <= maxi; ++i)
116     {
117         if((sockfd = client[i].fd) < 0)
118         {
119             continue;
120         }
121
122         if(client[i].revents & POLLIN)
123         {
124             if((n = read(sockfd, buf, sizeof(buf))) < 0)

```

```

125     {
126         if(errno == ECONNRESET)
127         {
128             printf("client[%d] abort connect\n", i);
129             close(sockfd);
130             client[i].fd = -1;
131         }
132         else
133         {
134             perror("read n = 0 error");
135         }
136     }
137     else if(n > 0)
138     {
139         for(j = 0; j < n; ++j)
140         {
141             buf[j] = toupper(buf[j]);
142         }
143         write(sockfd, buf, n);
144         write(STDOUT_FILENO, buf, n);
145     }
146     else
147     {
148         close(sockfd);
149         client[i].fd = -1;
150     }
151
152     if(--nready == 0)
153     {
154         break;
155     }
156 }
157 }
158 }
159 close(listenfd);
160 close(connfd);
161
162 return 0;
163 }

```

客户端

```

1  #include <stdio.h>
2  #include <sys/types.h>
3  #include <sys/socket.h>
4  #include <arpa/inet.h>
5  #include <unistd.h>
6  #include <ctype.h>
7  #include <string.h>
8  #include <stdlib.h>
9
10 #define SERV_IP "127.0.0.1"
11 #define SERV_PORT 8888
12
13 int main()
14 {
15     int cfd;

```



```

16     struct sockaddr_in serv_addr;
17     char buf[BUFSIZ];
18     int nByte;
19
20     cfd = socket(AF_INET, SOCK_STREAM, 0);
21     if(-1 == cfd)
22     {
23         perror("socket error");
24         exit(-1);
25     }
26
27     memset(&serv_addr, 0, sizeof(serv_addr));
28     serv_addr.sin_family = AF_INET;
29     serv_addr.sin_port = htons(SERV_PORT);
30     serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
31     /* inet_pton(cfd, SERV_IP, &serv_addr.sin_addr.s_addr); */
32
33     connect(cfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr));
34
35     while(1)
36     {
37         fgets(buf, sizeof(buf), stdin); //hello world ----> hello world\n\n0
38         write(cfd, buf, strlen(buf));
39
40         nByte = read(cfd, buf, sizeof(buf));
41
42         write(STDOUT_FILENO, buf, nByte);
43     }
44
45     close(cfd);
46     return 0;
47 }

```

4、epoll

4.1、接口解析

是Linux下IO多路复用接口select/poll的增强版本，能显著提高程序在大量并发连接中只有少量活跃的情况下的系统CPU利用率，因为它会复用文件描述符集合来传递结果而不是迫使开发者每次等待事件之前都必须重新准备要侦听的文件描述符集合，另一个原因是获取事件的时候，它无须遍历整个被侦听的描述符集，只要遍历哪些被内核IO事件唤醒而加入Ready队列的描述符集合就行了。

```

1  #include <sys/epoll.h>
2  int epoll_create(int size);
3  size: 参数size用来告知内核监听的文件描述符的个数，与内存大小有关。
4
5
6  //控制某个epoll监控的文件描述符上的事件：注册、修改、删除
7  int epoll_ctl(int epfd, int op, int fd, struct epoll_event *event);
8  epfd: epoll_create函数返回的值
9  op: EPOLL_CTL_ADD/EPOLL_CTL_MOD/EPOLL_CTL_DEL
10 fd: 将哪个文件描述符以op的方式加在以epfd建立的树上
11 event: 告诉内核需要监听的事情。
12 struct epoll_event
13 {

```

```

14     uint32_t    events;
15     epoll_data_t data;
16 };
17 typedef union epoll_data
18 {
19     void *ptr;
20     int    fd;
21     uint32_t u32;
22     uint64_t u64;
23 } epoll_data_t;
24
25 //等待所监控文件描述符上有事件的产生
26 int epoll_wait(int epfd, struct epoll_event *events, int maxevents, int
    timeout);
27 events: 用来存内核得到事件的集合（这里是个传出参数）
28 maxevents: 告知内核这个events有多大，这个maxevents的值不能大于创建epoll_create时的
    size
29 timeout: 是超时时间
30         -1: 阻塞
31         =0: 立即返回，非阻塞
32         >0: 指定毫秒
33 返回值: 成功返回有多少文件描述符就绪，时间到时返回0，出错返回-1

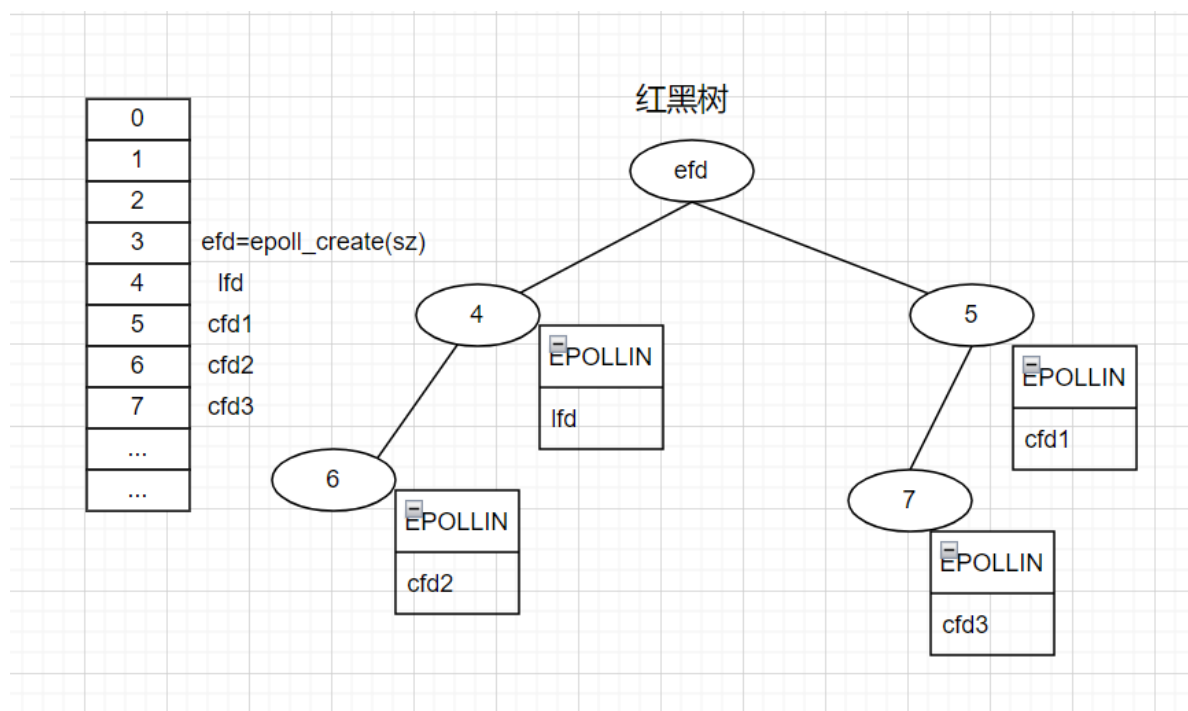
```

4.2、优缺点

优点:

- 1、文件描述符数目没有上限：通过epoll_ctl()来注册一个文件描述符，内核中使用红黑树的数据结构来管理所有需要监控的文件描述符。
- 2、基于事件就绪通知方式：一旦被监听的某个文件描述符就绪，内核会采用类似于callback的回调机制，迅速激活这个文件描述符，这样随着文件描述符数量的增加，也不会影响判定就绪的性能。
- 3、维护就绪队列：当文件描述符就绪，就会被放到内核中的一个就绪队列中，这样调用epoll_wait获取就绪文件描述符的时候，只要取队列中的元素即可，操作的时间复杂度恒为O(1)。

4.3、图解



4.4、类型区别

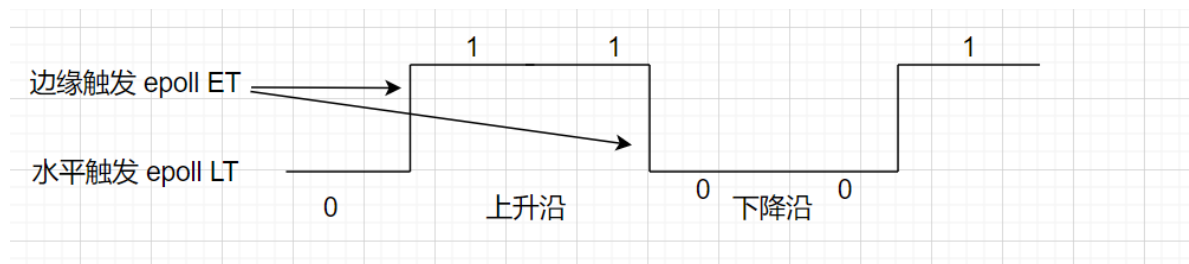
水平触发(level-triggered)

只要文件描述符关联的**读内核缓冲区**非空，有数据可以读取，就一直发出可读信号进行通知；当文件描述符关联的**内核写缓冲区**不满，有空间可以写入，就一直发出可写信号进行通知。LT模式支持阻塞和非阻塞两种方式。epoll默认的模式是LT。

边缘触发(edge-triggered)

当文件描述符关联的**读内核缓冲区由空转化为非空**的时候，则发出可读信号进行通知；当文件描述符关联的**内核写缓冲区由满转化为不满**的时候，则发出可写信号进行通知。

两者的区别在哪里呢？水平触发是只要读缓冲区有数据，就会一直触发可读信号，而边缘触发仅仅在空变为非空的时候通知一次。LT(level triggered)是缺省的工作方式，并且同时支持block和no-block socket。在这种做法中，内核告诉你一个文件描述符是否就绪了，然后你可以对这个就绪的fd进行IO操作。如果你不作任何操作，内核还是会继续通知你的，所以，这种模式编程出错误可能性要小一点。传统的select/poll都是这种模型的代表。

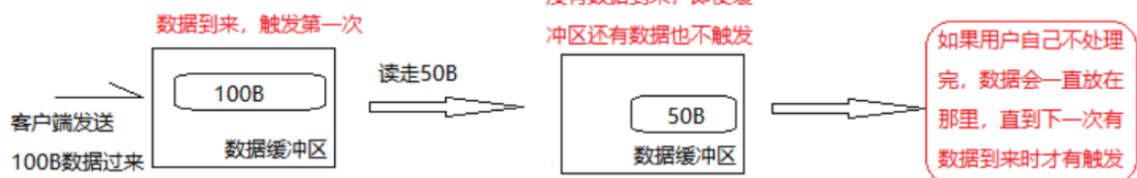


当设置了边缘触发以后，以可读事件为例，对“有数据到来”这事件为触发。

(1) 水平触发方式



(2) 边沿触发方式



select/poll/epoll除了应用于fd外，像管道、文件也是可以的。

4.5、代码实现（C语言）

server端

```
1 #include <stdio.h>
2 #include <sys/types.h>
3 #include <sys/socket.h>
4 #include <arpa/inet.h>
5 #include <sys/select.h>
6 #include <sys/time.h>
7 #include <sys/epoll.h>
8 #include <stdlib.h>
9 #include <strings.h>
10 #include <unistd.h>
```

```

11 #include <ctype.h>
12
13 #define SERV_PORT 8888
14 #define OPEN_MAX 5000
15
16 int main()
17 {
18     int listenfd, connfd, sockfd, epfd;
19     struct sockaddr_in serv_addr, clie_addr;
20     socklen_t clie_addr_len;
21     int ret, i, j, nready, nByte;
22     char buf[BUFSIZ], str[BUFSIZ];
23     struct epoll_event evt, ep[OPEN_MAX];
24
25
26     listenfd = socket(AF_INET, SOCK_STREAM, 0);
27     if(-1 == listenfd)
28     {
29         perror("socket error");
30         exit(-1);
31     }
32
33     int opt = 1;
34     setsockopt(listenfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
35
36     bzero(&serv_addr, sizeof(serv_addr));
37     serv_addr.sin_family = AF_INET;
38     serv_addr.sin_port = htons(SERV_PORT);
39     serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
40
41     ret = bind(listenfd, (struct sockaddr *)&serv_addr,
42 sizeof(serv_addr));
43     if(-1 == ret)
44     {
45         perror("bind error");
46         exit(-1);
47     }
48
49     ret = listen(listenfd, 128);
50     if(-1 == ret)
51     {
52         perror("listen error");
53         exit(-1);
54     }
55
56     epfd = epoll_create(OPEN_MAX);
57     if(-1 == epfd)
58     {
59         perror("epoll_create error");
60         exit(-1);
61     }
62
63     evt.events = EPOLLIN;
64     evt.data.fd = listenfd;
65
66     ret = epoll_ctl(epfd, EPOLL_CTL_ADD, listenfd, &evt);
67     if(-1 == ret)
68     {

```

```

68     perror("epoll_ctl error");
69     exit(-1);
70 }
71
72 while(1)
73 {
74     nready = epoll_wait(epfd, ep, OPEN_MAX, -1);
75     if(nready < 0)
76     {
77         perror("select error");
78         exit(-1);
79     }
80
81     for(i = 0; i < nready; ++i)
82     {
83         if(!(ep[i].events & EPOLLIN))
84         {
85             continue;
86         }
87
88         if(ep[i].data.fd == listenfd) //如果是连接事件
89         {
90             clie_addr_len = sizeof(clie_addr);
91             connfd = accept(listenfd, (struct sockaddr *)&clie_addr,
&clie_addr_len);
92             if(-1 == connfd)
93             {
94                 perror("accept error");
95                 exit(-1);
96             }
97             printf("receive from %s from port %d\n",
98                 inet_ntop(AF_INET, &clie_addr.sin_addr, str,
sizeof(str)),
99                 ntohs(clie_addr.sin_port));
100
101             evt.events = EPOLLIN;
102             evt.data.fd = connfd;
103             epoll_ctl(epfd, EPOLL_CTL_ADD, connfd, &evt);
104         }
105         else //不是连接建立事件，而是读写事件(信息传递事件)
106         {
107             sockfd = ep[i].data.fd;
108             nByte = read(sockfd, buf, sizeof(buf));
109             if(nByte == 0)
110             {
111                 ret = epoll_ctl(epfd, EPOLL_CTL_DEL, sockfd, NULL);
112                 if(-1 == ret)
113                 {
114                     perror("epoll_ctl error");
115                 }
116                 close(sockfd);
117                 printf("client[%d] closed connection\n", sockfd);
118             }
119             else if(nByte < 0)
120             {
121                 perror("epoll_ctl error");
122                 ret = epoll_ctl(epfd, EPOLL_CTL_DEL, sockfd, NULL);
123                 if(-1 == ret)

```

```

124         {
125             perror("epoll_ctl error");
126         }
127         close(sockfd);
128
129     }
130     else
131     {
132         for(j = 0; j < nByte; ++j)
133         {
134             buf[j] = toupper(buf[j]);
135         }
136         write(sockfd, buf, nByte);
137         write(STDOUT_FILENO, buf, nByte);
138     }
139 }
140 }
141 }
142
143 close(listenfd);
144 close(connfd);
145
146 return 0;
147 }

```

客户端

```

1  #include <stdio.h>
2  #include <sys/types.h>
3  #include <sys/socket.h>
4  #include <arpa/inet.h>
5  #include <unistd.h>
6  #include <ctype.h>
7  #include <string.h>
8  #include <stdlib.h>
9
10 #define SERV_IP "127.0.0.1"
11 #define SERV_PORT 8888
12
13 int main()
14 {
15     int cfd;
16     struct sockaddr_in serv_addr;
17     char buf[BUFSIZ];
18     int nByte;
19
20     cfd = socket(AF_INET, SOCK_STREAM, 0);
21     if(-1 == cfd)
22     {
23         perror("socket error");
24         exit(-1);
25     }
26
27     memset(&serv_addr, 0, sizeof(serv_addr));
28     serv_addr.sin_family = AF_INET;
29     serv_addr.sin_port = htons(SERV_PORT);
30     serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);

```

```
31  /* inet_pton(cfd, SERV_IP, &serv_addr.sin_addr.s_addr); */
32
33  connect(cfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr));
34
35  while(1)
36  {
37      fgets(buf, sizeof(buf), stdin); //hello world ----> hello world\n\0
38      write(cfd, buf, strlen(buf));
39
40      nByte = read(cfd, buf, sizeof(buf));
41
42      write(STDOUT_FILENO, buf, nByte);
43  }
44
45  close(cfd);
46  return 0;
47 }
```